

第7章 图

7.1 图的定义和术语

7.2 图的存储结构

7.3 图的遍历

7.4 最小生成树

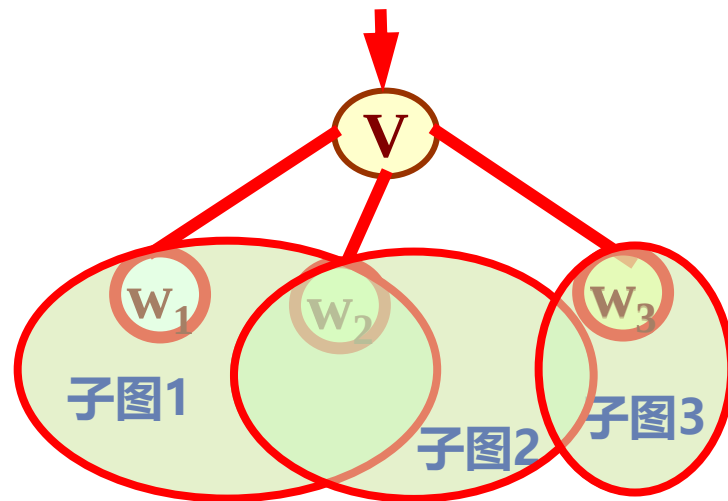
7.5 活动网络

7.6 最短路径

回顾

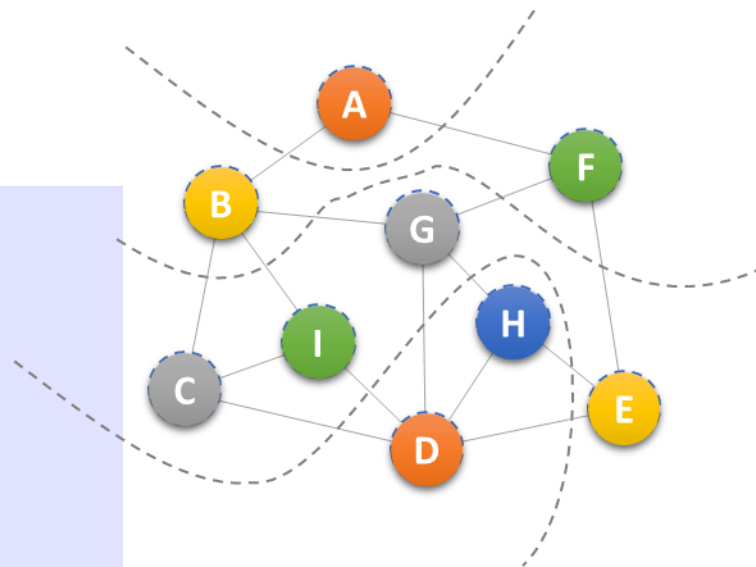
□ 深度优先遍历

1. 访问顶点 V ;
2. **for** (W_1 、 W_2 、 W_3)
 若该邻接点 W_i 未被访问，
 则深度优先搜索遍历该子图。



□ 广度优先遍历

1. 访问顶点 V ;
2. **for** (W_1 、 W_2 、 W_3)
 若该邻接点 W_i 未被访问，则访问 W_i
3. 若 W_i 的邻接点未被访问，则访问之
4.。



回顾

例，已知一个图的邻接矩阵如下，则从顶点A出发按深度优先遍历和广度优先遍历，可能得到的一种顶点序列分别为：

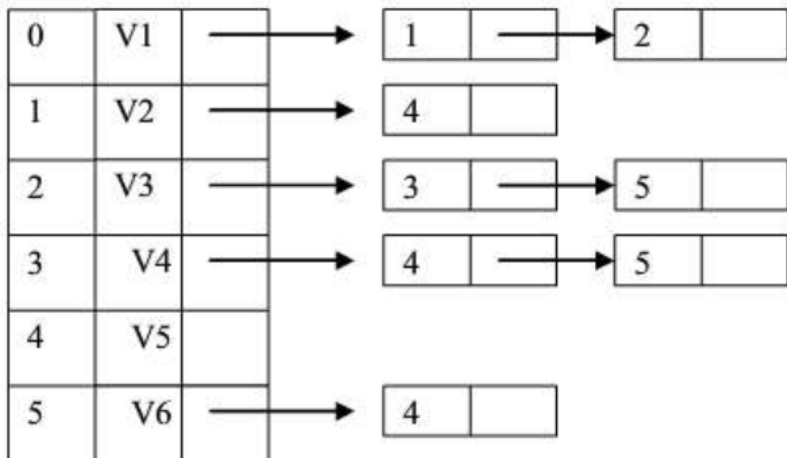
	A	B	C	D	E	F
A	0	1	1	0	1	0
B	1	0	0	1	0	0
C	1	0	0	0	1	1
D	0	1	0	0	1	0
E	1	0	1	1	0	1
F	0	0	1	0	1	0

深度优先遍历：ABDECF

广度优先遍历：ABCEDF

回顾

例，已知一个图的邻接矩阵如下，则从顶点V1出发按深度优先遍历和广度优先遍历，可能得到的一种顶点序列分别为：



深度优先遍历： $V_1V_2V_5V_3V_4V_6$

广度优先遍历： $V_1V_2V_3V_5V_4V_6$

第7章 图

7.1 图的定义和术语

7.2 图的存储结构

7.3 图的遍历

7.4 最小生成树

7.5 活动网络

7.6 最短路径

7.4.1 图的连通性问题

连通图

在无向图中，若从顶点 v_i 到顶点 v_j 有路径，则称顶点 v_i 与 v_j 是连通的。如果图中任意一对顶点都是连通的，则称此图是连通图。

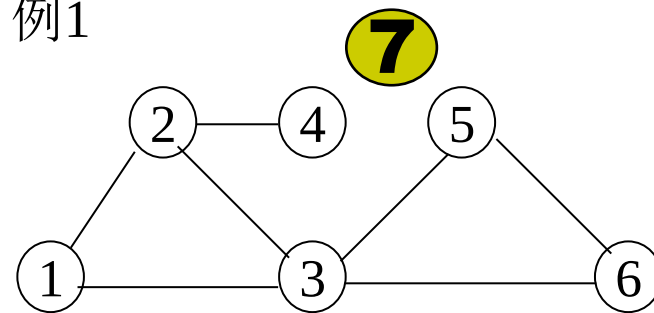
非连通图的极大连通子图叫做连通分量。

强连通图

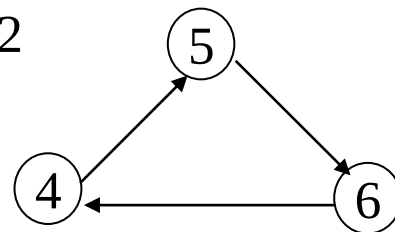
在有向图中，若对于每一对顶点 v_i 和 v_j ，都存在一条从 v_i 到 v_j 和从 v_j 到 v_i 的路径，则称此图是强连通图。

非强连通图的极大强连通子图叫做强连通分量。

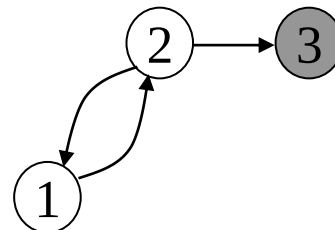
例1



例2

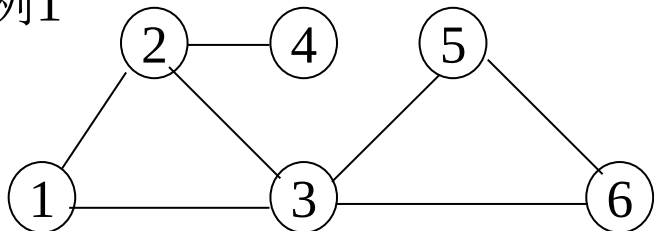


例3

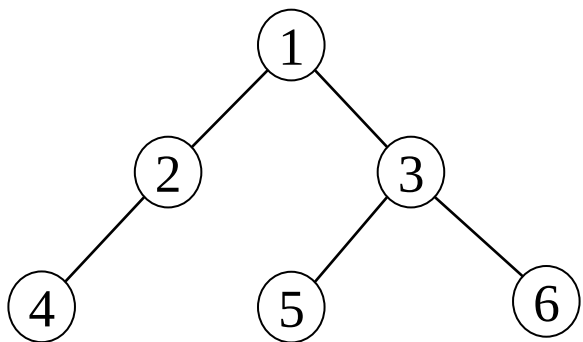
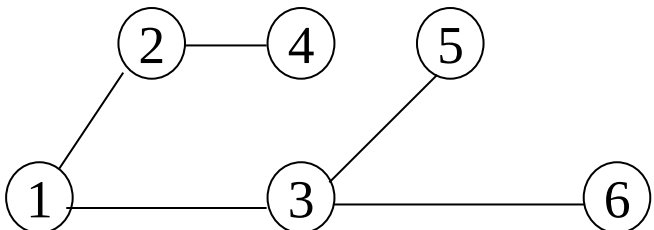


7.4.1 图的连通性问题

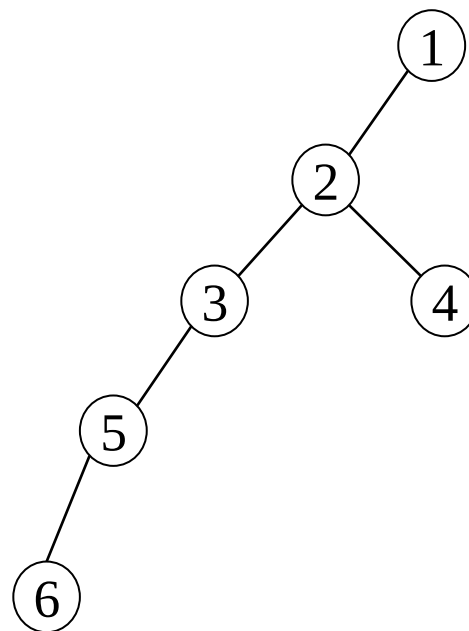
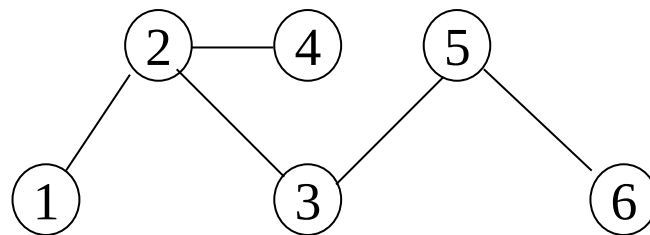
例1



广度优先遍历: 1 2 3 4 5 6



深度优先遍历: 1 2 3 5 6 4



7.4.1 图的连通性问题

思考1：对连通图进行遍历，得到的是什么？

——得到的将是一个极小连通子图，即图的**生成树**！

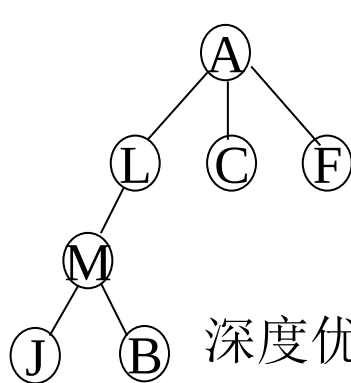
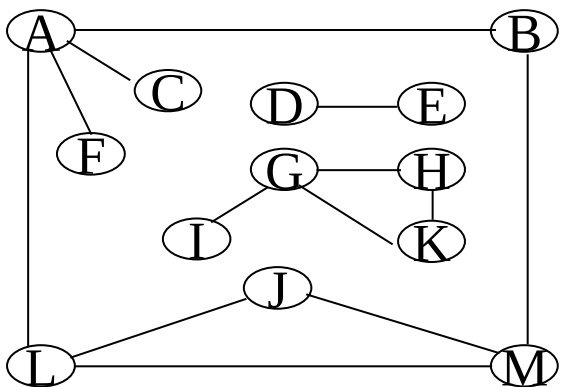
由**深度优先搜索**得到的生成树，称为**深度优先搜索生成树**。

由**广度优先搜索**得到的生成树，称为**广度优先搜索生成树**。

思考2：对非连通图进行遍历，得到的是什么？

——得到的将是各连通分量的生成树，即图的**生成森林**！

例



深度优先生成森林

7.4.1 图的连通性问题

□ 连通图的生成树:

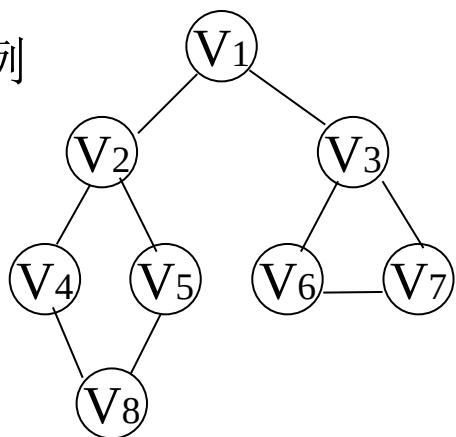
是连通图的一个极小连通子图，它含有图中**全部顶点**，但只有 **$n-1$** 条边。

□ 非连通图的生成森林:

由若干棵**生成树**组成，含图中全部顶点，但构成这些树的边是最少的。

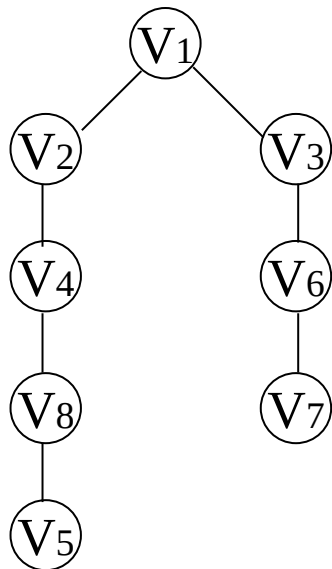
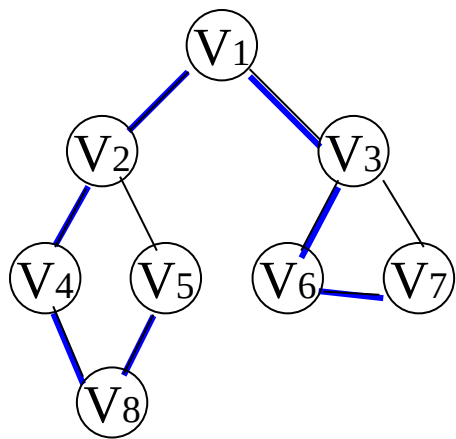
7.4.1 图的连通性问题

例

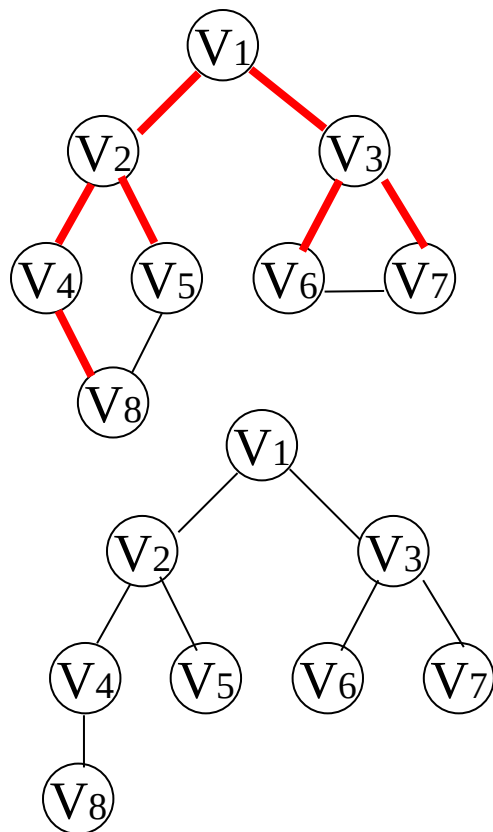


深度遍历: $V1 \Rightarrow V2 \Rightarrow V4 \Rightarrow V8 \Rightarrow V5 \Rightarrow V3 \Rightarrow V6 \Rightarrow V7$

广度遍历: $V1 \Rightarrow V2 \Rightarrow V3 \Rightarrow V4 \Rightarrow V5 \Rightarrow V6 \Rightarrow V7 \Rightarrow V8$



深度优先生成树

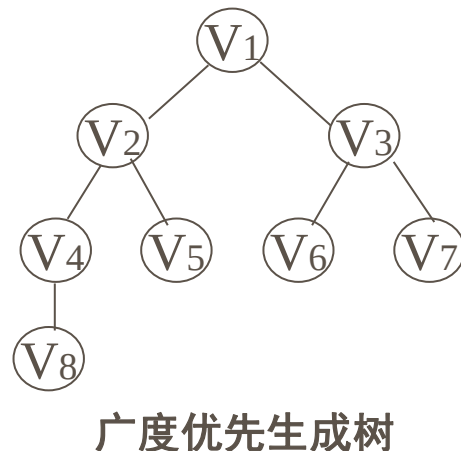
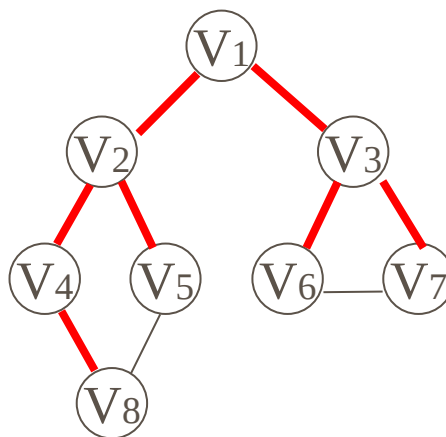
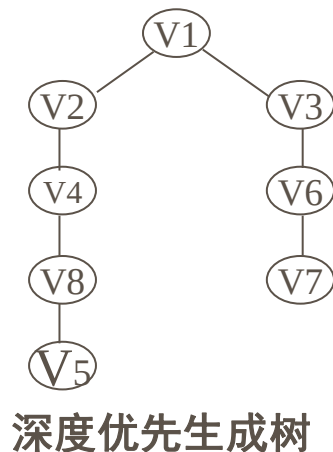
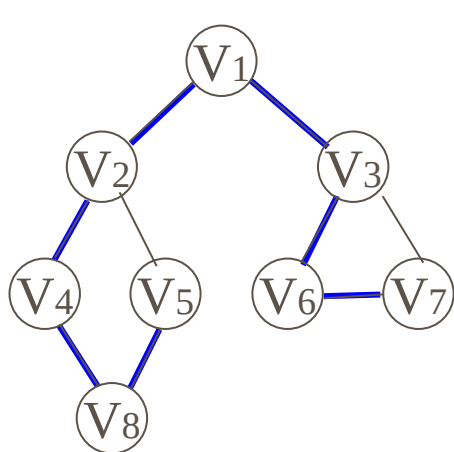


广度优先生成树

说明: 连通图的生成树不唯一

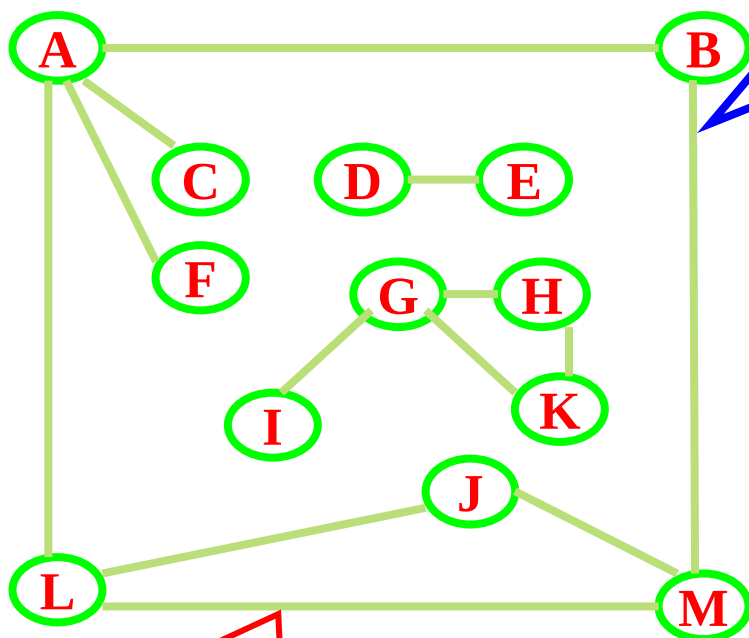
7.4.1 图的连通性问题

- 所有生成树具有以下共同特点：
 - 生成树的顶点个数与图的顶点个数相同（是连通图的极小连通子图）
 - 一个有 n 个顶点的连通图的生成树只有 $n-1$ 条边
 - 在生成树中再加一条边必然形成回路
 - 生成树中任意两个顶点间的路径是唯一的
 - 从不同顶点出发或搜索次序不同，可得到不同的生成树



7.4.1 图的连通性问题

画出下图的生成森林（或极小连通子图）



这是一个无向非连通图
(参见教材P170-171例)

生成森林的定义（对有向或无向图均适用）：是若干棵生成树的集合，含全部顶点，但构成这些树的边或弧是最少的。

求解步骤：

Step1:先求出邻接矩阵或邻接表；

Step2:写出DFS或BFS结果序列；

Step3:画出对应子图或生成森林。

其实由邻接矩阵或邻接表
也能直接画出生成森林

下面选用邻接表方式来求深度优先搜索生成森林

7.4.1 图的连通性问题

1、先写出邻接表（或邻接矩阵）

2、再写出DFS结果

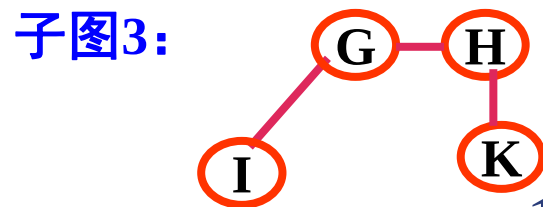
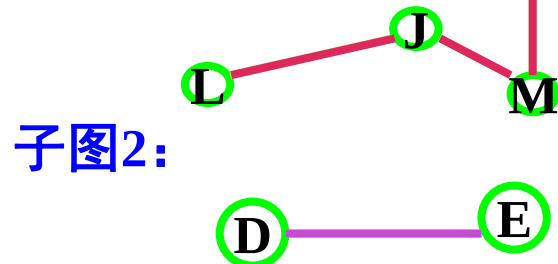
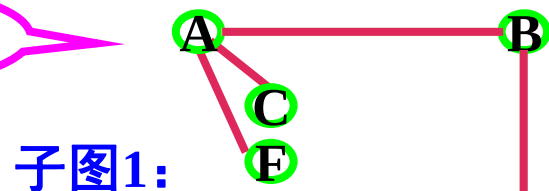
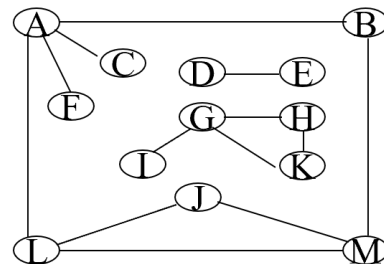
ABMJLCF **DE** **GHKI**

3、画出对应子图或生成森林。

7.4.1 图的连通性问题

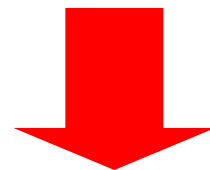
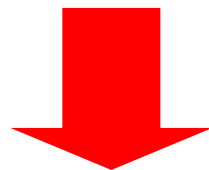
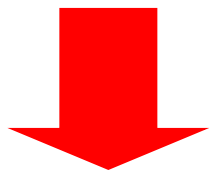
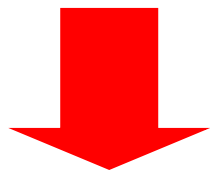
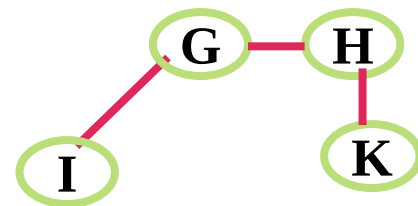
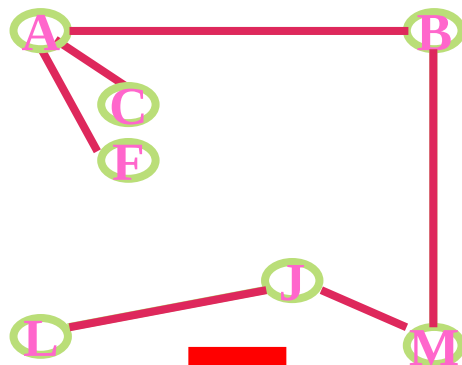


最小连通!

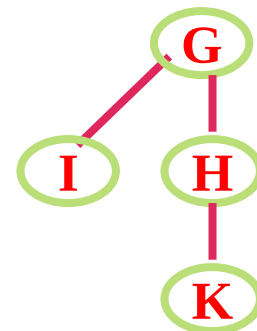
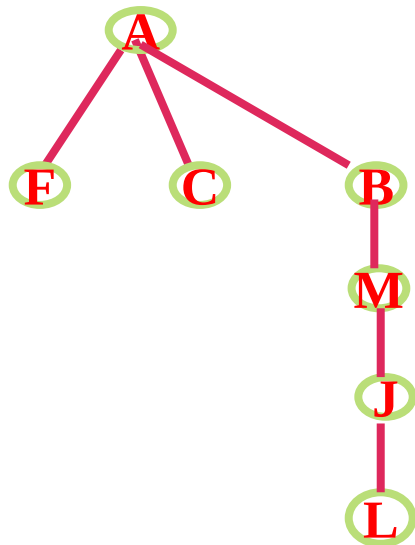


7.4.1 图的连通性问题

子图
(或连通分量)



生成森林



7.4.1 图的连通性问题

图生成树

```
void DFSTree(Graph G, int v, CSTree &T){
    visited[v] = true;
    bool first = true; //标记是否第一个兄弟
    CSNode *pre;
    for(int w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w)){
        if(!visited[w]){
            CSNode *p = (CSNode*)malloc(sizeof(CSNode)); //分配孩子结点
            *p = {getValue(G,w), NULL, NULL};
            if(first){ //w 是v 的第一个未被访问的邻接顶点
                T->firstChild = p; first = false;
            }
            else pre->nextSibling = p; //w是v的其他未被访问的邻接顶点
            pre = p;
            DFSTree(G,w,p); //从第w个顶点出发深度优先遍历图G，建立子生成树q
        }
    }
}
```

```
typedef char TElemType;
typedef struct CSNode{
    TElemType data;
    CSNode *firstChild, *nextSibling;
}CSNode,*CSTree;
```

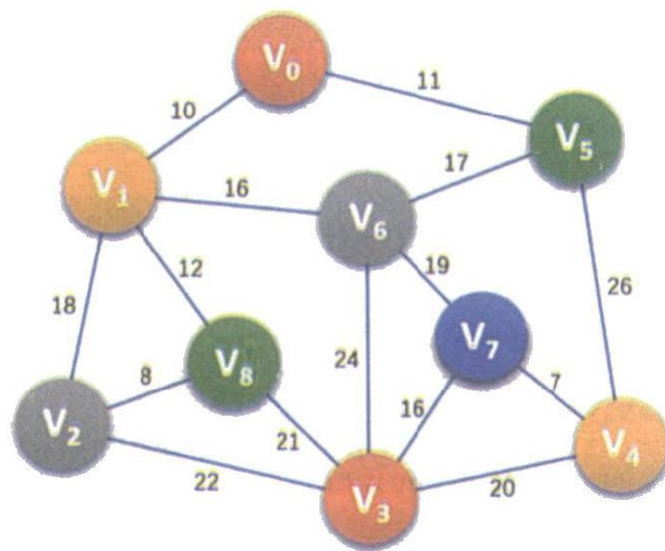

7.4.1 图的连通性问题

```
void DFSForest(Graph G, CSTree &T){
    T = NULL; //生成森林的根
    bool visited[MAXSIZE]; //标记
    for(int i = 0; i < G.vexnum; i++) visited[i] = false;
    CSNode pre; //保存前一个兄弟
    for(int v = 0; v < G.vexnum; v++){
        if(!visited[v]){
            CSNode* p = (CSNode*)malloc(sizeof(CSNode)); //创建结点
            *p = {getValue(G,v), NULL, NULL}; //赋值
            if(!T) T = p; //v是生成森林的第一棵子树
            else pre->nextSibling = p; //后一棵树做前棵的兄弟
            pre = p; //第一次后赋值
            DFSTree(G, v, p); //以p为根创建其子树
        }
    }
}
```

```
typedef char TElemType;
typedef struct CSNode{
    TElemType data;
    CSNode *firstChild, *nextSibling;
}CSNode, *CSTree;
```

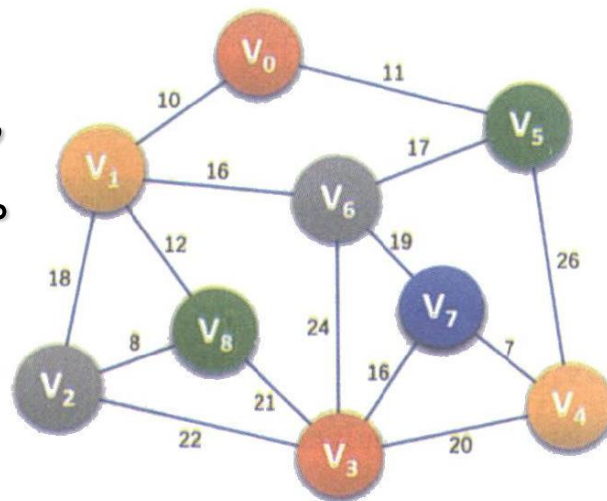
7.4.2 最小生成树

假设电信工程师要为一个镇的九个村庄架设通信网络做设计，村庄位置大致如下图，其中 $v_0 \sim v_8$ ，是村庄，之间连线的数字表示村与村间的可通达的直线距离，现要求必须用最小的成本完成这次任务。**怎么办？**

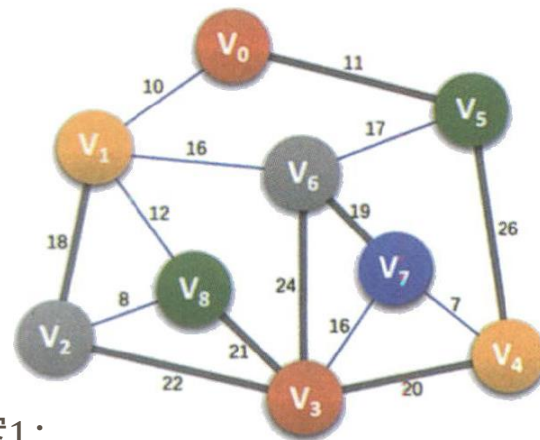
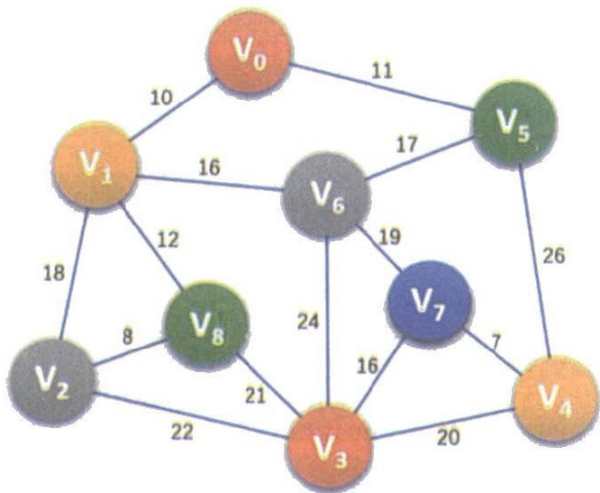


7.4.2 最小生成树

- 问题演化
 - 要在 n 个村庄间建立通信联络网，
 - 顶点——表示村庄
 - 边的权——村庄间建立通信线路所需花费代价
- 问题分析
 - n 个村庄间，最多可设置 $n(n-1)/2$ 条线路
 - n 个村庄间建立通信网，只需 $n-1$ 条线路
 - 如何在可能的线路中选择 $n-1$ 条，能把所有村庄（顶点）均连起来，且总耗费（各边权值之和）最小。

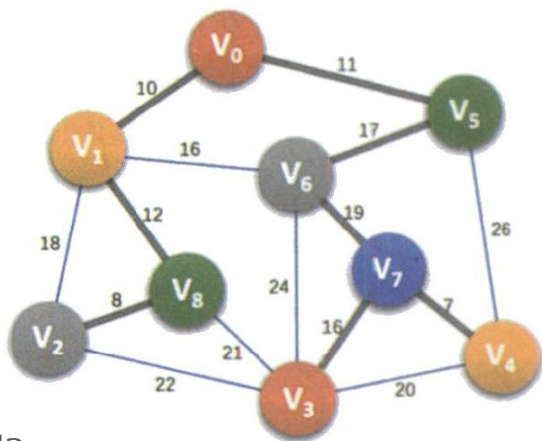


7.4.2 最小生成树



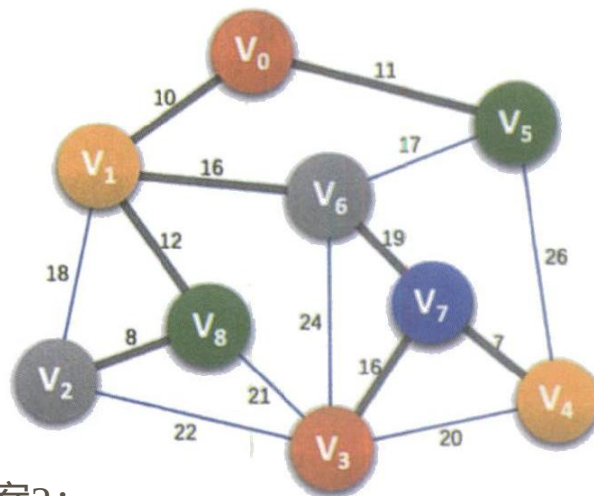
方案1:

$$18 + 22 + 20 + 26 + 11 + 21 + 24 + 19 = 161$$



方案2:

$$8 + 12 + 10 + 11 + 17 + 19 + 16 + 7 = 100$$



方案2:

$$8 + 12 + 10 + 11 + 16 + 19 + 16 + 7 = 99$$

7.4.2 最小生成树

该问题等价于：

在 n 个顶点的连通网中，构造网的一棵**最小生成树**，即：在 e 条带权的边中选取 $n-1$ 条边(不构成回路)，使“权值之和”为最小。

找最经济通讯网 $\longrightarrow$ 求最小生成树

7.4.2 最小生成树

- **生成树的权**：对连通网络来说，边附上权，生成树也带权，我们把生成树**各边的权值总和**称为**生成树的权**
- **最小代价生成树**：在一个连通网的所有生成树中，各边的**代价之和最小**的那棵生成树称为该连通网的**最小代价生成树**（Minimum Cost Spanning Tree），简称为**最小生成树(MST)**。

7.4.2 最小生成树

如何求最小生成树?

两种方法: { **Prim**算法
Kruskal算法

最小生成树有如下重要性质 (MST性质) :

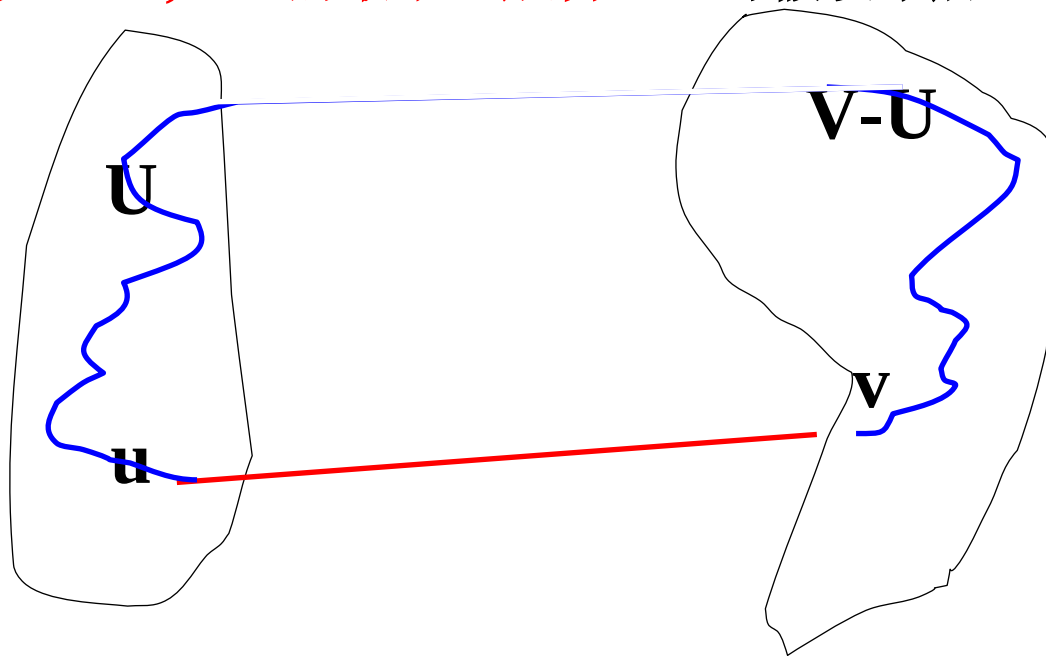
设 $N=(V, \{E\})$ 是一连通网, U 是顶点集 V 的一个非空子集。
若 (u, v) 是一条具有**最小权值的边**, 其中 $u \in U, v \in V-U$,
则**存在一棵包含边 (u, v) 的最小生成树**。

换言之: 连通网的**最小生成树的 $n-1$ 边中**一定包含连通网中**权值最小的边**。(例如 在1-10 中选择3个数, 使得这三个数之和 最小, 则这三个数中必定包含1。)

7.4.2 最小生成树

我们用反证法来证明这个**MST**性质：

假设不存在这样一棵包含边 (u, v) 的最小生成树。任取一棵最小生成树 T ，将 (u, v) 加入 T 中。根据树的性质，此时 T 中必形成一个包含 (u, v) 的回路，且回路中必有一条边 (u', v') 的权值大于或等于 (u, v) 的权值。删除 (u', v') ，则得到一棵代价小于等于 T 的生成树 T' ，且 T' 为一棵包含边 (u, v) 的最小生成树。这与假设矛盾，故该性质得证。



7.4.2 最小生成树

普利姆（Prim）算法

设： $N = (V, E)$ 是个连通网，
另设 U 为最小生成树的顶点集， TE 为最小生成树的边集。

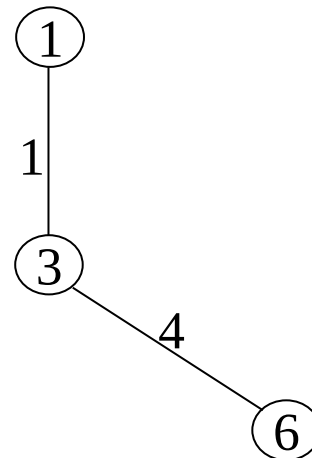
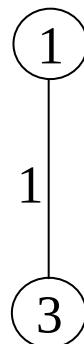
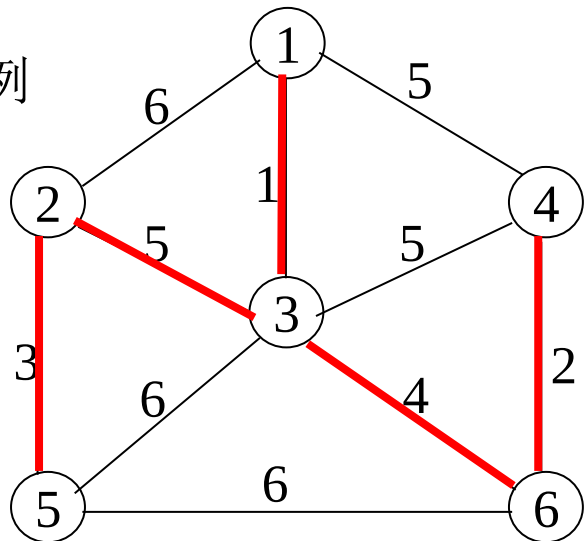
构造步骤：

- (1) 初始状态： $U = \{u_0\}$, ($u_0 \in V$) , $TE = \varnothing$,
- (2) 在 $u \in U, v \in V - U$ 所有的边 $(u, v) \in E$ 中，找一条代价最小的边 (u_0, v_0) ，并将边 (u_0, v_0) 并入集合 TE ，同时 v_0 并入 U 。
- (3) 重复（2），直到 $U = V$ 为止。

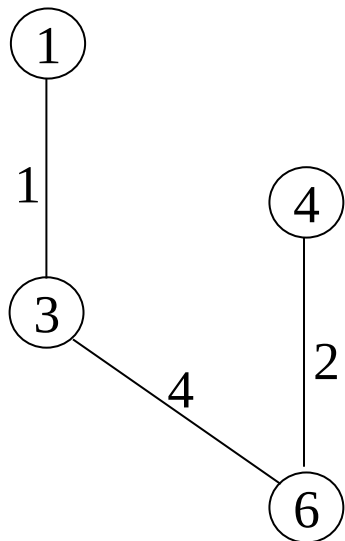
此时 TE 中必有 $n-1$ 条边， $T = (V, \{TE\})$ 就是最小生成树。

[注]：在最小生成树的生成过程中，所选的边都是一端在U中，另一端在V-U中。选最小边的过程是一个向集合U中添加顶点的过程。

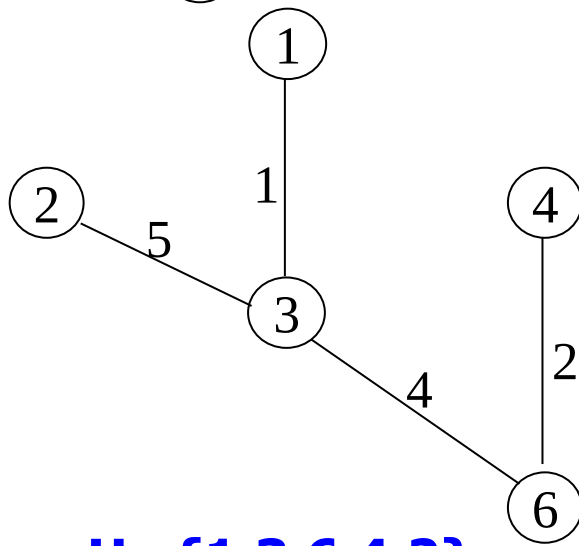
例



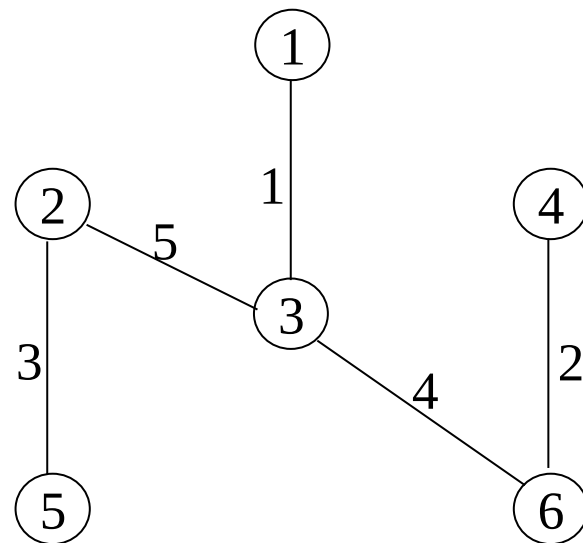
$U=\{1\}$ $U=\{1,3\}$ $U=\{1,3,6\}$



$U=\{1,3,6,4\}$



$U=\{1,3,6,4,2\}$

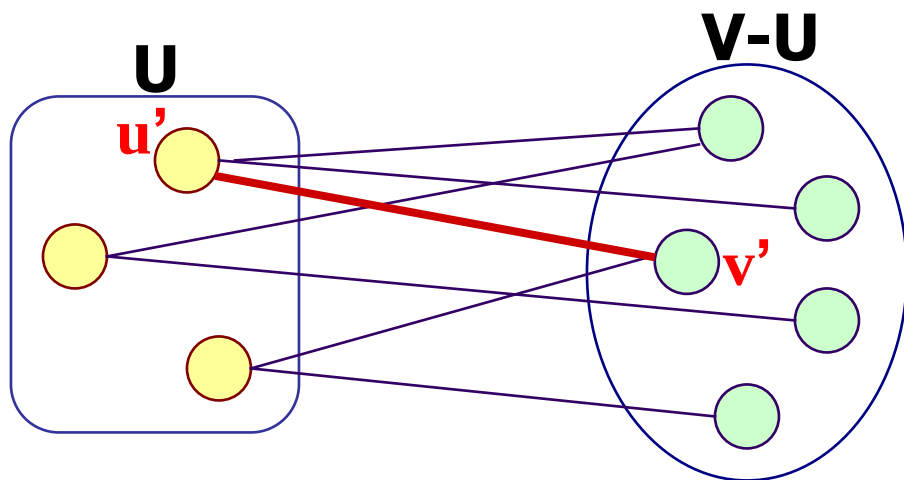


$U=\{1,3,6,4,2,5\}$

7.4.2 最小生成树

一般情况下所添加的顶点应满足下列条件:

在生成树的构造过程中, 图中 n 个顶点分属两个集合: **已落在生成树上的顶点集 U** 和 **尚未落在生成树上的顶点集 $V-U$** , 则应在所有连通 U 中顶点 u 和 $V-U$ 中顶点 v 的边中选取**权值最小的边 (u', v')** , 并把 v' 添加到 U 中。



7.4.2 最小生成树

计算机内怎样实现Prim（普里姆）算法？

Prime算法特点：将顶点归并，与边数无关，适于稠密网。

设计思路：故采用邻接矩阵作为图的存储表示。

① 增设一辅助数组Closededge[n]，以记录从U到V-U具有最小代价的边，每个数组分量都有两个域：



要求：使Closededge[i].lowcost = $\min((u, v_i)) \quad u \in U$

```
struct {
```

```
    VertexType adjvex; // u在U集中的顶点序号
```

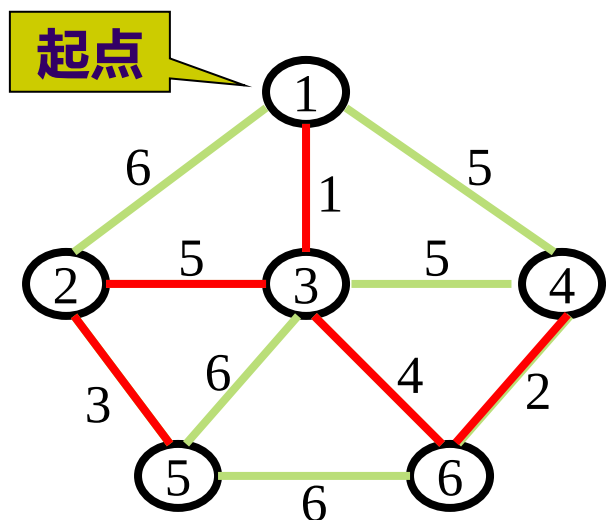
```
    VRType lowcost; // 边的权值
```

```
} closedge[MAX_VERTEX_NUM];
```

u 从U ($u_1 \sim u_n$) 中挑选，选择U中到顶点 v_i 边的权值最小的顶点

7.4.2 最小生成树

具体示例：

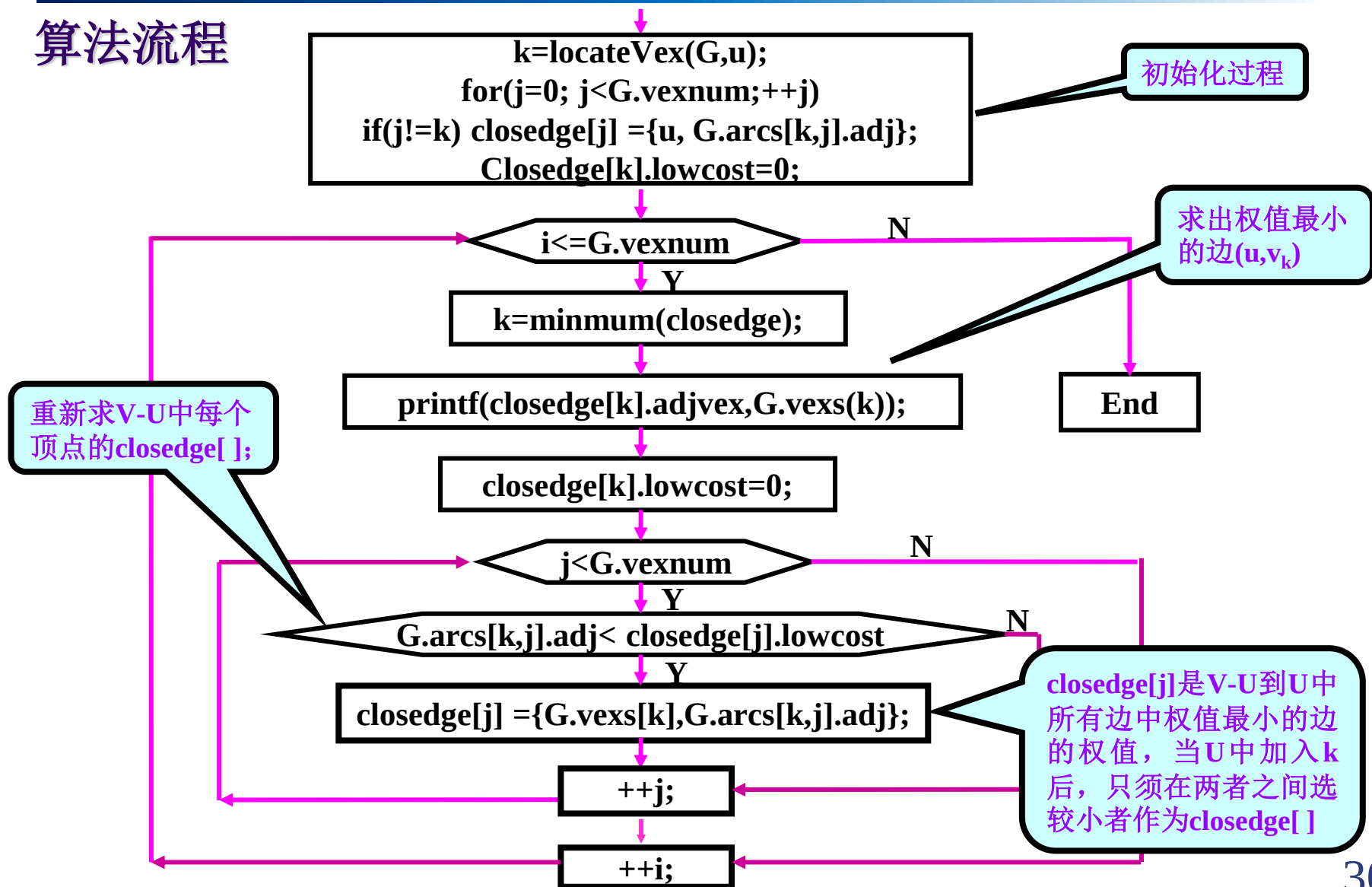


1	0	6	1	5	∞	∞
2	6	0	5	∞	3	∞
3	1	5	0	5	6	4
4	5	∞	5	0	∞	2
5	∞	3	6	∞	0	6
6	∞	∞	4	2	6	0

$\begin{matrix} v \\ \text{closed edge} \end{matrix}$	2	3	4	5	6	U	V-U
adjvex	1	1	1	1	1	{1}	{2,3,4,5,6}
lowcost	6	1	5	∞	∞		
adjvex	3		1	3	3	{1, 3}	{2, 4, 5, 6}
lowcost	5	0	5	6	4		
adjvex	3		6	3		{1,3,6}	{2, 4, 5}
lowcost	5	0	2	6	0		
adjvex	3			3		{1,3,6,4}	{2, 5}
lowcost	5	0	0	6	0		
adjvex				2		{1,3,6,4,2}	{5}
lowcost	0	0	0	3	0		
adjvex						{1,3,6,4,2,5}	{}
lowcost	0	0	0	0	0		

7.4.2 最小生成树

算法流程



7.4.2 最小生成树

Prim (普里姆) 算法描述

```
void MiniSpanTree_P(MGraph G, VertexType u)
{ //用普里姆算法从顶点u出发构造网G的最小生成树T,输出T的各条边
  k = LocateVex ( G, u ); //找顶点u在图中的位置k
  for ( j=0; j<G.vexnum; ++j ) // 辅助数组初始化
    if (j!=k) //u到各顶点的权值 {adjvex, lowcost} (u, vj)
      closedge[j] = { u, G.arcs[k][j].adj };
  closedge[k].lowcost = 0; // 初始, U = {u}, 即将u并入U
  .....
  for (i=1; i<G.vexnum; ++i) //以其边权值大小, 选择其余顶点并入
  {
    继续向生成树上添加顶点; //即查找下一条最小权值边 (u, vi) ,
    //把该边所依附的另一个顶点vi加入到U中。
  }
} // MiniSpanTree_P
```

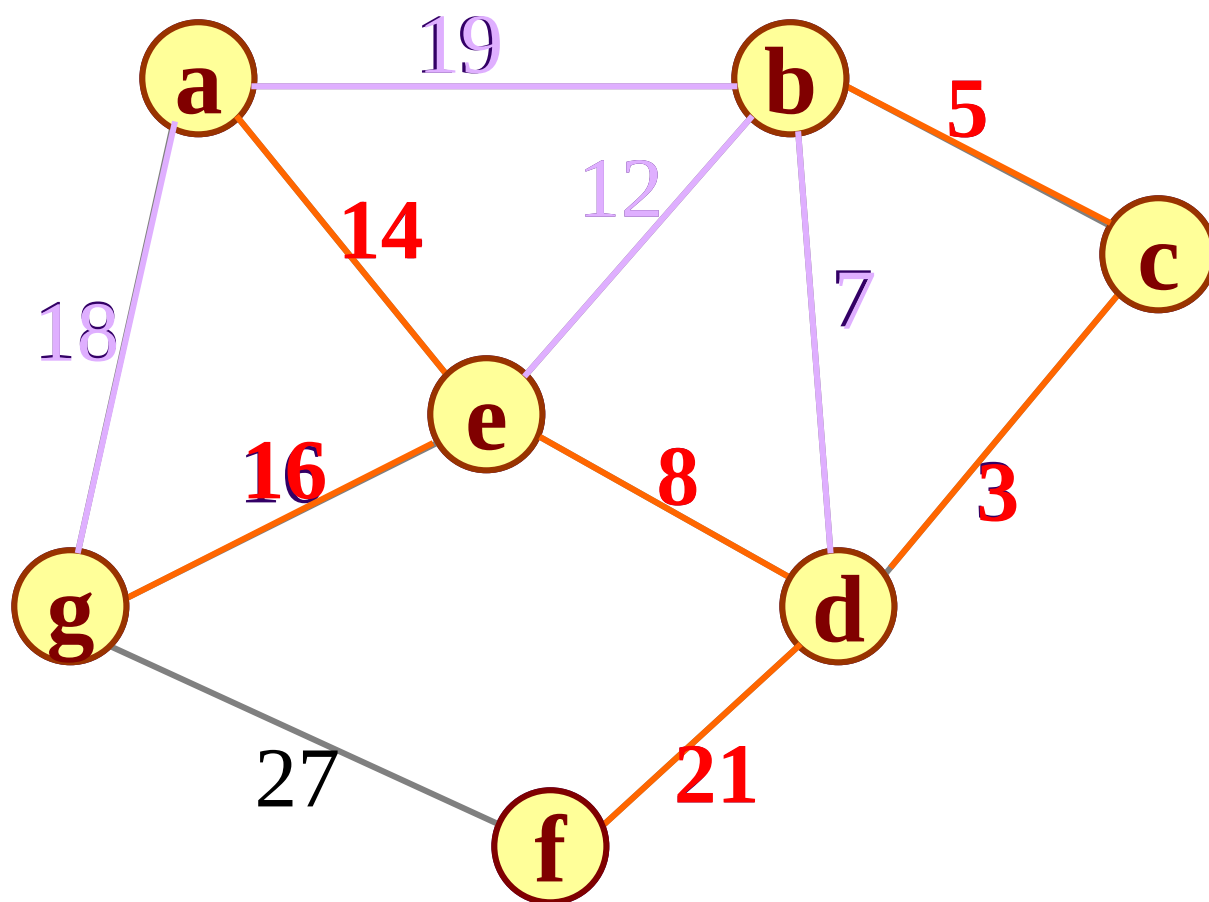
7.4.2 最小生成树

```
( for (i=1; i<G.vexnum; ++i){}循环中内容)
k = minimum(closedge); // 求出加入生成树的下一个顶点(k)
printf(closedge[k].adjvex, G.vexs[k]);

// 输出生成树上一条边 ( $u, v_k$ ) , 其中G.vexs是顶点向量
closedge[k].lowcost = 0; // 把第k个顶点并入U集
for (j=0; j<G.vexnum; ++j)
    // 新顶点并入U后, (可能) 需要修改U中原顶点到V-U中
    其它顶点的最小边
    //为第k个顶点到V-U中其它顶点的最小边
if (G.arcs[k][j].adj < closedge[j].lowcost)
    closedge[j] = { G.vexs[k], G.arcs[k][j].adj };
```


7.4.2 最小生成树

例如:



7.4.2 最小生成树

怎样实现Kruskal算法？

特点：将边归并——适于求稀疏网的最小生成树。
故采用邻接表作为图的存储表示。

设计思路：

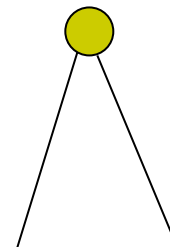
① 设每条边对应的结构类型为：

	v_i	v_j	weight	
--	-------	-------	--------	--

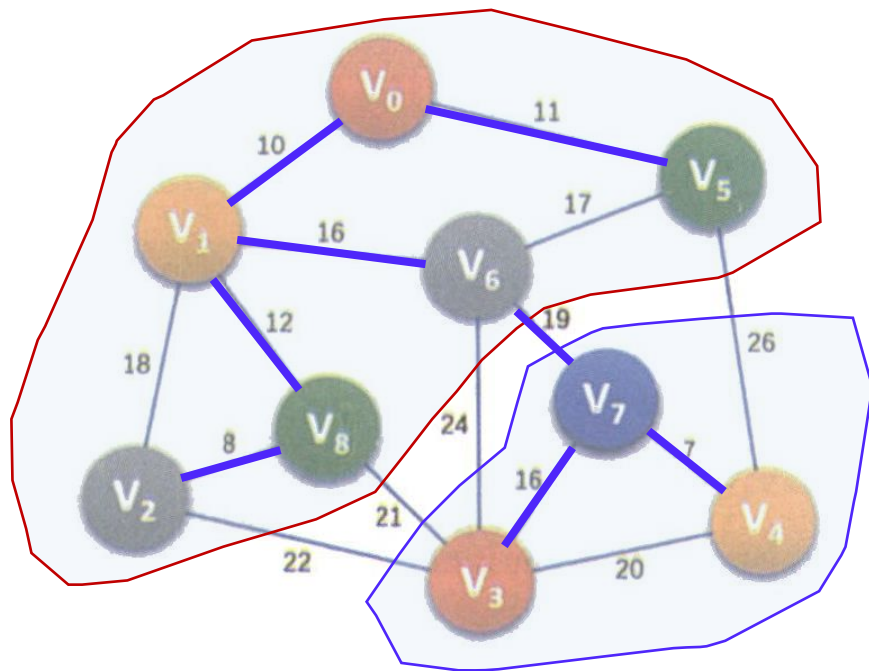
初态：按权值排序(以堆排序为佳，堆顶即为权值最小的边)

- ② 取堆顶元素，加入到对应最小生成树的新邻接表中（初始为空），从堆中删除它并重新排序堆，每次耗时 $\log_2(e)$ ；
- ③ 重复上一步，注意每次加入时要判断是否“多余”（即是否已被新表中的连通分量包含）；
- ④ 直到堆空。

显然，Kruskal算法的时间效率 = $O(e \log_2 e)$



7.4.2 最小生成树



<i>begin</i>	<i>end</i>	<i>weight</i>
4	7	7
2	8	8
0	1	10
0	5	11
1	8	12
3	7	16
1	6	16
5	6	17
1	2	18
6	7	19
3	4	20
3	8	21
2	3	22
3	6	24
4	5	26

	<i>parent</i>
0	1
1	5
2	8
3	7
4	7
5	8
6	7
7	0
8	6

7.4.2 最小生成树

```
void MiniSpanTree_Kruskal(MGraph G){
    int i, r1, r2;
    Edge edges[MAXEDGE];
    int parent[MAXVEX];
    /* 此处省略将邻接矩阵G转化为边集数组edges并按权由小到大排序的代码*/
    for (i = 0; i < G.numVertexes; i++) parent[i] = 0;
    for (i = 0; i < G.numEdges; i++){
        r1 = Find(parent, edges[i].begin);
        r2 = Find(parent, edges[i].end);
        if (r1 != r2) {
            parent[r1] = r2;
            printf("(%d, %d) %d\n", edges[i].begin, edges[i].end, edges[i].weight);
        }
    }
}
```

```
int Find(int *parent, int f) {
    while ( parent[f] > 0) {
        f = parent[f];
    }
    return f;
}
```

```
typedef struct {
    int begin;
    int end;
    int weight;
} Edge;
```

7.4.2 最小生成树

Kruskal算法

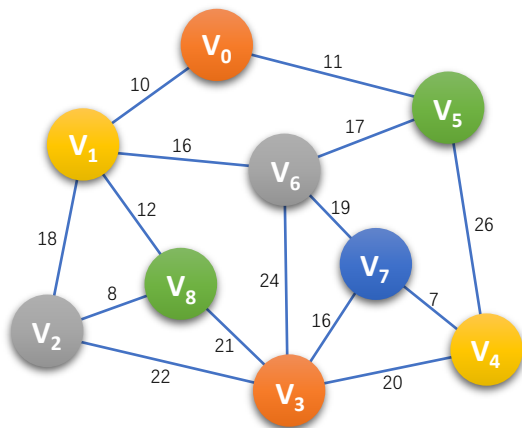
```
void MiniSpanTree_Kruskal(ELGraph G, SqList& MSTree)
{// G.edge 中依权值从小到大存放有向网中各边，按克鲁斯卡尔
// 算法求得生成树的边存放在顺序表 MSTree 中
    MFSet F;
    InitSet(F, G.vexnum);           // 将森林F初始化为n棵树的集合
    InitList(MSTree, G.vexnum);      // 初始化生成树为空
    i=0; k=1;
    while( k<G.vexnum ) {
        e = G.edge[i];              // 取第 i 条权值最小的边
        r1 = fix_mfset(F, LocateVex(e.begin));
        r2 = fix_mfset(F, LocateVex(e.end)); // 返回两个顶点所在树的树根
        if (r1 != r2) {              // 选定生成树上第k条边
            if (ListInsert(MSTree, k, e)) k++; // 插入生成树
            mix_mfset(F, r1, r2);         // 将两棵树归并为一棵树
        } // if
        i++; // 继续考察下一条权值最小边
    } // while
    DestroySet(F);
} // MiniSpanTree_Kruskal
```

```
typedef struct {
    VertexType begin;
    VertexType end;
    VRType weight;
}EdgeType;
typedef ElemType EdgeType;
// 有向网的定义
typedef struct {
    // 顶点信息
    VertexType
        vexs[MAX_VERTEX_NUM];
    // 边的信息
    EdgeType
        edge[MAX_EDGE_NUM];
    // 图中顶点的数目和边的数目
    int vexnum,arcnum;
}ELGraph;
```

7.4.2 最小生成树

破圈法

- 1.在图中寻找环，并将访问过的环顶点标记为已访问，若已经访问则不进入递归。
- 2.寻找过程中记录当前环中的权重最大边，及其对应信息。
- 3.寻找环的过程闭合之后，去除最大边。
- 4.重复步骤1-3，直到图中不存在环为止。



begin	end	weight
4	7	7
2	8	8
0	1	10
0	5	11
1	8	12
3	7	16
1	6	16
5	6	17
1	2	18
6	7	19
3	4	20
3	8	21
2	3	22
3	6	24
4	5	26

7.4.2 最小生成树

比较两种算法

算法名

普里姆算法

克鲁斯卡尔算法

时间复杂度

$O(n^2)$

$O(e \log e)$

适应范围

稠密图

稀疏图

7.4.2 最小生成树

例：下列关于最小生成树的叙述中，正确的是（ ）

1. 最小生成树的代价唯一
2. 所有权值最小的边一定会出现在所有的最小生成树中
3. 使用普里姆（**Prim**）算法从不同顶点开始得到的最小生成树一定相同
4. 使用普里姆算法和克鲁斯卡尔（**Kruskal**）算法得到的最小生成树总不同

正在答疑
